CodeQuery API: Technical Report

Sachin Sindhe
COMP3011 – Web Services and Web Data

GitHub Repository: Sachin-1712/RepoLens
API Documentation: View PDF Documentation
Presentation Slides: Download slides.pptx

Abstract. This report describes CodeQuery, a database-backed RESTful API that ingests Git repositories and supports natural-language questions about the codebase using Retrieval-Augmented Generation (RAG). The system stores repositories, analysis artefacts, and question history in PostgreSQL, performs semantic retrieval using pgvector, and optionally generates grounded answers via a local LLM (Ollama). A "Lightweight Mode" runs by default (API + database only), ensuring storage-safe operation on consumer hardware and graceful degradation when advanced services are unavailable.

1 Introduction and Objectives

CodeQuery is an AI-powered code intelligence API designed to ingest Git repositories and help users understand unfamiliar codebases by asking questions such as "How is routing implemented?" or "Where is authentication handled?". The system follows a Retrieval-Augmented Generation (RAG) workflow: repository files are chunked and embedded, relevant code excerpts are retrieved for a given question, and an answer is generated from that evidence (optionally using a local LLM).

The project objectives were:

- Build a RESTful API meeting coursework requirements: multiple CRUD endpoints, persistent SQL storage, correct HTTP status codes, JSON responses, and OpenAPI (Swagger) documentation.
- Implement repository ingestion: clone a repository, discover and filter files, chunk code, and store structured chunks in the database.
- Provide repository Q&A using semantic retrieval over stored chunks, with an optional local LLM for natural-language answer generation.
- Support **Lightweight Mode** by default to run reliably on limited storage (under 20GB) without paid APIs or heavyweight background services, while gracefully degrading when optional components are not enabled.

2 Technology Stack and Design Rationale

2.1 Python and FastAPI

Python 3.12 was selected due to its strong ecosystem for repository manipulation and semantic tooling. FastAPI was chosen because it provides:

- **Asynchronous support** for I/O-heavy operations (database queries, HTTP calls).
- **Pydantic validation** for typed request/response schemas, improving robustness and reducing runtime errors.

- **Automatic OpenAPI documentation** via Swagger UI (`/docs`) and ReDoc (`/redoc`), supporting both assessment documentation and viva demonstrations.

2.2 Database: PostgreSQL with pgvector

A core design decision was to use PostgreSQL extended with `pgvector` rather than introducing a separate vector database. This provides:

- **Unified datastore** for relational entities (repositories, questions, analysis metadata) and vector embeddings (code chunks).
- **Relational integrity** with foreign keys linking repositories, chunks, and questions, improving traceability and auditability.
- **Efficient similarity search** (cosine distance) directly within SQL for top-k retrieval during RAG.

SQLAlchemy 2.0 (async) was used as the ORM, with Alembic for migrations.

2.3 AI Models: Local Embeddings and Optional Generation

To avoid paid APIs by default and support offline usage:

- **Embeddings:** HuggingFace sentence-transformers `all-MiniLM-L6-v2` produces 384-dimensional embeddings for code chunks.
- **Optional LLM:** Ollama can be enabled to run a small local model (`mistral`) for grounded answer generation.

The system is designed so that Lightweight Mode does not require the LLM. When the LLM is unavailable, the Q&A endpoint returns a clear service-unavailable outcome but still provides retrieved sources as evidence.

2.4 Containerisation and Lightweight Mode

The application is managed via Docker Compose. By default, only `api` and `db` start in Lightweight Mode. Optional profiles enable advanced components:

- `--profile async` enables Celery + Redis for background ingestion.
- `--profile llm` enables Ollama for local generation.

This ensures a deterministic, storage-safe default while allowing richer functionality when hardware permits.

3 System Architecture

3.1 Overview

CodeQuery uses a layered architecture:

- **API layer (FastAPI):** routes for repositories, questions, and analysis.
- **Service layer:** ingestion, chunking, embedding, and Q&A engine.
- **Persistence layer (PostgreSQL + pgvector):** stores repositories, code chunks (with vectors), questions, and analysis state.

3.2 Key Resources and Endpoints

The API follows resource-based REST design. Table 1 summarises core endpoints.

Method	Endpoint	Description
POST	`/api/v1/repositories`	Add repository for analysis (triggers ingestion).
GET	`/api/v1/repositories`	List repositories.
GET	`/api/v1/repositories/{id}`	Get repository details.
PUT	`/api/v1/repositories/{id}`	Update / re-analyse repository.
DELETE	`/api/v1/repositories/{id}`	Delete repository and related artefacts.
POST	`/api/v1/repositories/{id}/questions`	Ask a question about a repository.
GET	`/api/v1/repositories/{id}/questions`	List question history for a repository.
GET	`/api/v1/questions/{id}`	Retrieve a single question record.
DELETE	`/api/v1/questions/{id}`	Delete a question record.
GET	`/api/v1/repositories/{id}/analysis`	View analysis / ingestion status.
GET	`/api/v1/repositories/{id}/statistics`	Retrieve code statistics for a repository.
GET	`/api/v1/health`	Health check.

Table 1: Core CodeQuery API endpoints (as documented in the project README).

4 Repository Ingestion Pipeline

4.1 Clone and File Discovery

When a repository is added, the system clones it and discovers files while filtering irrelevant directories (e.g., `node_modules`, `.git`) to reduce noise. Lightweight Mode includes a storage warning and recommends testing with small repositories (e.g., `octocat/Hello-World`) to prevent excessive disk usage.

4.2 Chunking

The ChunkingService aims to preserve semantic units where possible. Python files are chunked using AST parsing to keep functions/classes intact, while other languages fall back to generic chunking. Each chunk stores metadata (file path and line spans) to support traceable citations.

4.3 Embedding and Storage

Each chunk is embedded using `all-MiniLM-L6-v2` and stored in PostgreSQL with its vector representation. This enables semantic search within SQL using pgvector similarity queries.

5 Repository Q&A and RAG Workflow

5.1 Retrieval

When a question is posted, the system embeds the query and retrieves the top-k most similar code chunks from pgvector. Retrieval is scoped to the target repository, ensuring evidence comes from the correct codebase.

5.2 Augmented Generation

If the LLM profile is enabled, a strict prompt containing the retrieved chunks and the user query is sent to the local model via Ollama to generate a grounded answer. The response is returned alongside sources that cite file paths and line spans.

5.3 Graceful Fallback in Lightweight Mode

If the LLM is unavailable (Lightweight Mode), the Q&A engine does not crash. Instead, it returns a clear outcome indicating generation is unavailable (HTTP 503) while still returning the retrieved evidence. This preserves usefulness as a semantic code-search tool while keeping the system storage-safe by default.

5.4 Persistence and Auditability

All Q&A interactions are persisted with answer text, confidence score, model used, processing time, and retrieved sources. This supports auditability (what was answered and based on which evidence) and allows users to review question history.

6 Testing Strategy

Automated testing was implemented using `pytest`. The test suite covers:

- **API tests:** health checks, repository CRUD behaviours, correct handling of validation errors (422) and missing resources (404).
- **Service tests:** chunking behaviour and ingestion file discovery rules.
- **Reproducibility:** tests can be executed inside the container (`docker compose exec api pytest -q`) for examiner-friendly verification.

Testing improves reliability and demonstrates professional practice aligned with REST API development standards.

7 Challenges and Lessons Learned

- **Balancing ambition with deployability:** A full RAG stack can become resource-heavy. Implementing Lightweight Mode with optional Docker profiles was critical to keep the default footprint small and reliable.
- **Async database management:** Async SQLAlchemy improves throughput but requires careful session lifecycle handling to avoid connection leaks and inconsistent transaction states.
- **Chunking trade-offs:** Larger chunks preserve context but reduce retrieval precision, while smaller chunks increase precision but can lose meaning. AST-based chunking for Python demonstrated the value of language-aware segmentation.

8 Limitations and Future Work

8.1 Limitations

- **Repository size constraints:** Large repositories can increase ingestion time and disk usage on consumer hardware.
- **Local model performance:** CPU-only local generation is slower than hosted inference; quality is model-dependent.
- **Language-aware chunking coverage:** AST-based chunking is strongest for Python; other languages currently rely more on generic strategies.

8.2 Future Work

- **Security hardening:** add API key authentication, rate limiting for expensive endpoints (ingestion and Q&A), and strict allowlisting/validation of repository URLs to mitigate SSRF-style risks.
- **Scalability & job control:** enhance analysis jobs with progress percentages, cancellation, retries, and idempotency (e.g., avoid re-indexing the same commit SHA).
- **Language-aware intelligence:** extend AST-based chunking beyond Python (TypeScript/Java/Go) and add symbol indexing (classes/functions) to improve retrieval precision and citations.
- **Richer analysis outputs:** add endpoints for dependency graphs, change summaries across commits, and lightweight security heuristics (e.g., hard-coded secrets, risky imports) stored as first-class findings.
- **Deployment readiness:** add structured logging, metrics (request latency, job duration), and optional auth/RBAC to support safe public deployment.

9 Generative AI Declaration

In accordance with University guidance, generative AI tools (specifically Google Gemini / Antigravity Agent) were used for creative exploration and advanced problem-solving, aligning with the highest-level usage band:

- **Concept Exploration & Alternative Architectures:** GenAI was used extensively to evaluate and pivot between different design choices, such as contrasting a pure FastAPI setup versus incorporating Celery for async ingestion, and designing the innovative "Lightweight Mode" fallback to ensure the system is deployable on constrained hardware without crashing.
- **Data Structure & RAG Prototyping:** AI tools assisted in designing the PostgreSQL `pgvector` schema and prototyping the semantic retrieval pipeline, allowing me to focus on curating the high-level logic and cutting-edge database architecture.
- **Responsibility:** While AI guided the creative conception of cutting-edge architectural patterns, all final design decisions, API routing, data modelling, and the `pytest` validation suite were verified and synthesised independently to meet coursework objectives.

These advanced architectural explorations firmly ground the project in modern, state-of-the-art backend practices. Thoughtful analysis of this usage is evident in the included logs. Examples of exported conversation logs demonstrating this creative, high-level structural usage are attached as supplementary material.

A Generative AI Conversation Logs

(genai-conversation-logs.pdf)

GenAI Conversation Excerpts

Development and runtime

- **Python 3.12** – backend language
- **FastAPI** – REST API framework (OpenAPI generation)
- **Uvicorn** – ASGI server to run FastAPI
- **Pydantic (v2)** – request/response validation and settings
- **SQLAlchemy 2.0 (async)** – ORM + async DB access
- **Alembic** – database migrations
- **PostgreSQL** – relational database
- **pgvector** – vector extension for similarity search inside PostgreSQL

Background processing (optional advanced mode)

- **Celery** – background task queue for ingestion/analysis jobs
- **Redis** – Celery broker/backing service

AI and code intelligence

- **GitPython** – clone and interact with Git repos programmatically
- **sentence-transformers (HuggingFace)** – embeddings model library
- **all-MiniLM-L6-v2** – embedding model (384-dim) used for chunk vectors
- **Ollama** – local LLM inference server (optional mode)

- **qwen2.5:0.5b** (or your chosen model) – LLM used for answer generation (optional)

Testing and quality

- **pytest** – unit/integration testing framework
- **FastAPI TestClient / httpx** – API-level testing (depending on your setup)

Containerisation and reproducibility

- **Docker** – container runtime
- **Docker Compose** – multi-service orchestration (API, DB, worker, Redis, Ollama)

Documentation and delivery

- **Swagger UI / OpenAPI** – interactive API documentation at `/docs` (auto-generated by FastAPI)
- **Google Docs** – writing the final technical report
- **Google Slides** – viva presentation slides
- **Browser "Print to PDF"** – exporting Swagger docs and report to PDF

Visual assets for slides (optional but used for polish)

- **Shots.so** – framed, polished screenshots (Swagger, GitHub, etc.)
- **Ray.so** (or **Carbon.now.sh**) – pretty code/JSON/terminal snippet images

- **Excalidraw** (or **Draw.io**) – architecture/pipeline diagrams

- **Nano Banana Pro (Google Slides "Help me visualise")** – generating slide visuals/diagrams/images

GenAI assistance

- **OpenAI ChatGPT**
- **Google Gemini**
- **Claude**

The following excerpts document how generative AI was used as a support tool during this coursework (planning, debugging, documentation). I reviewed and verified all outputs, and the final implementation was tested and finalised by me.

Included excerpts:

1. Project scope and REST resource design

2. Docker Compose and local deployment checks

3. Testing strategy and viva preparation

Help me brainstorm ideas for building a Restful API,

Thought for a few seconds

Here are **resume-worthy REST API project ideas** that fit the COMP3011 brief (CRUD + SQL DB + 4+ endpoints + proper status codes + docs + deploy) and also give you "talking points" for the oral exam.
I'm also biasing these toward your current strengths (FastAPI/Next.js/MCP, deployment, data work) so it naturally complements your CV.

---

1) “InternOps” — Internship & Application Tracker + Analytics

Why it pops on a CV: practical, personal, analytics endpoints, clean CRUD, auth.

- **Core CRUD models:** Applications, Companies, Interviews, Notes
- **Analytics endpoints:** success-rate by stage, response-time stats, “next actions” dashboard, weekly progress
- **Extra polish:** JWT auth + per-user data isolation, pagination/filtering, rate limiting
- **Great demo moment:** show “pipeline view” + analytics charts in Swagger

---

2) “GitHub Sprint Insights API” — Repo Activity → Team Productivity Metrics

Why it pops: looks like real engineering tooling; perfect to discuss tradeoffs.

- **Data source:** GitHub API (pull requests, issues, commits)
- **Core CRUD:** Teams, Repos, Sprint configs, Goals
- **Analytics:** lead time, PR throughput, issue aging, contributor summaries
- **Extra:** background jobs (cron) to sync GitHub → SQL; caching layer; webhooks
- **Oral exam win:** justify data modelling + aggregation queries

---

3) “UK Transport Reliability API” — Delay Patterns + User-Reported Incidents

This is literally aligned with the brief’s example themes (transport analytics).

- **Core CRUD:** routes, stations, incidents, user reports
- **Analytics:** reliability score per route, delay heatmaps by hour/day, “top disruption causes”
- **Extra:** geo queries (nearest stations), validation, robust error codes

---

4) “Smart Recipe Cost + Nutrition API (UK)” — Recipes, Allergens, Basket Pricing

Why it pops: tangible value; shows data integration + computed endpoints.

- **Data source:** open nutrition dataset + supermarket pricing snapshot (CSV)
- **Core CRUD:** recipes, ingredients, dietary tags, user pantry
- **Computed endpoints:** calories/macros, allergen warnings, estimated weekly cost, “swap suggestions”
- **Extra:** OpenAPI docs with examples, input validation, unit conversion

---

5) “RoommateSplit” — Bills, Debts, Settlements (Algorithmic Endpoint)

Why it pops: simple to demo, but the “settle up” algorithm looks impressive.

- **Core CRUD:** households, members, expenses, payments
- **Key endpoint:** `POST /settlements/compute` → minimal transactions to settle
- **Extra:** role-based access, audit log, idempotency keys (nice industry detail)

---

6) “Study Planner API” — Timetable + Revision Streaks + Exam Countdown

Why it pops: personal + analytics; easy to build quickly; great demo visuals.

- **Core CRUD:** modules, sessions, tasks, resources
- **Analytics:** streaks, workload balance, “risk score” for upcoming assessments
- **Extra:** reminders as scheduled jobs, export to iCal, tags + full-text search

---

7) “Dataset Explorer API” — Upload Dataset → Profile → Query → Insights

Directly matches the coursework’s “use AI for dataset discovery/import scripts” workflow.

- **Core CRUD:** datasets, tables, schemas, saved queries
- **Endpoints:** data profiling (nulls, histograms, top values), sampling, schema inference
- **Extra:** generates SQL import scripts + stores provenance (great for report discussion)

---

8) “MCP-Ready Local Agent API” — API that can be called as tools (Very Good+)

The brief explicitly mentions MCP compatibility as an “advanced feature” route.

- Build a normal REST API **plus** an `/mcp/tools` endpoint (or MCP server wrapper) exposing:
  - `create_record`, `search_records`, `run_analytics`, `generate_report`
- **Why it pops:** modern, “agentic” angle, aligns with your KPMG experience

---

3 “high mark” features you can bolt onto *any* idea

These map cleanly to the rubric (testing, docs, deployment, design justification).

1. **Auth + roles**
   JWT + user-scoped rows; optional admin role.

2. **Serious docs**
   OpenAPI/Swagger with example requests/responses + error codes; export to PDF.
3. **Analytics endpoints + tests**
   At least 2–3 non-trivial aggregated endpoints + pytest (unit + integration).

My top 2 picks for your resume (based on your CV)

- **GitHub Sprint Insights API** (looks closest to real “developer tooling”, matches your agentic/GitHub automation story)
- **Dataset Explorer API** (perfect for oral exam: data modelling, SQL, analytics, AI-assisted import scripts)

If you tell me which direction you like (tooling vs data/analytics vs lifestyle app), I’ll turn it into a **1-page project spec**: models, endpoints list, database schema outline, and a stack suggestion that’s easiest to build + defend in the viva.

ok based off this idea: "AI-Powered Code Intelligence & Search API: 2. AI-Powered Code Intelligence & Search API MCP-compatible GitHub/GitLab code search with semantic understanding What it does: FastAPI backend exposing endpoints for: Semantic code search (find code by intent, not keywords) Code explanation endpoint (explain what functions do) Vulnerability detection (analyze code for security issues) Code similarity detection (find duplicated logic) Dependency graph generation Agentic AI component: Agent analyzes codebases and builds knowledge graphs Learns from repository structure and commit history Provides intelligent recommendations CRUD Operations: Repositories (add/remove repos to analyze) Code snippets (save/annotate important code) Analyses (store historical code reviews) Vulnerabilities (track security findings) Why this stands out: Combines your ML background with web services MCP-compatible = cutting-edge Practical for your final year project (SAM2 segmentation) Shows understanding of modern AI architectures " what if i want to build an API where instead of searching the code, it analyses the repo and users should be able to ask questions about the repo. Does this qualify as a RESTful API as asked for our coursework? I

know this is a good idea and theres a possibility of me extending this project to make it better later but yeah. Look at this instagram reel below for more info on the idea I mentioned above.
https://www.instagram.com/reel/DUb-ECLj9XY/

Thought for 22s

Yes, this absolutely can qualify as a RESTful API for COMP3011, as long as you still build it around *resources* (stored in an SQL database) with proper CRUD, and the “ask questions” part is implemented as a resource-driven workflow rather than just one magic chatbot endpoint. The coursework only requires at least one data model with full CRUD linked to a database, at least four HTTP endpoints, JSON responses, correct status codes, and a working demo (local hosting or even MCP) plus docs and report.

Why “Repo Q&A” still counts as REST

A REST API is not “must be a search API”. It is “HTTP endpoints that act on resources”. Your resources can be:

- repositories
- ingestions (indexing runs)
- documents/chunks
- questions
- answers
- findings (vulnerabilities, TODOs, hotspots)

If you model it like that, it is a normal data driven REST API, with an AI powered feature on top. The brief even rewards advanced features such as MCP compatibility, and explicitly allows GenAI as a core part of the solution.

The Instagram reel idea you linked is basically “paste a GitHub repo link, build RAG, ask questions”. That is perfectly compatible with this approach.

The safe way to structure it so it clearly meets the rubric

Here is a clean resource model that ticks every requirement.

Core SQL models (simple and defendable in the viva)

1. **Repository**
- id, provider (github/gitlab), url, default_branch, created_at
2. **IngestionJob**
- id, repo_id, status (queued/running/done/failed), commit_sha, started_at, finished_at, error
3. **DocumentChunk**
- id, repo_id, file_path, start_line, end_line, content, embedding_vector_id (or store embedding separately)
4. **Question**
- id, repo_id, question_text, asked_at
5. **Answer**
- id, question_id, answer_text, citations_json, created_at

That gives you very obvious CRUD, proper DB integration, and a clear story.

Example endpoints that look RESTful and impressive

Repositories CRUD (meets minimum requirement)

- `POST /repos` create repo
- `GET /repos` list repos (with pagination)
- `GET /repos/{repoId}` get repo
- `PATCH /repos/{repoId}` update repo metadata
- `DELETE /repos/{repoId}` remove repo

Analysis workflow (very defendable)

- `POST /repos/{repoId}/ingestions` start indexing a repo at a commit
- `GET /repos/{repoId}/ingestions/{jobId}` check status and logs
- `GET /repos/{repoId}/documents` list indexed files/chunks (filter by file_path)

Repo Q&A (the headline feature)

- `POST /repos/{repoId}/questions` create a Question resource
- `GET /repos/{repoId}/questions/{questionId}` fetch question

- `GET /repos/{repoId}/questions/{questionId}/answer` fetch Answer resource (or return answer immediately if synchronous)

This makes “ask questions” a normal REST interaction: you create a question, you retrieve an answer.

How to keep it genuinely RESTful (and avoid examiner pushback)

Common pitfall: making everything “POST /ask” and nothing else.

Instead:

- Treat indexing as a job resource (async friendly).
- Treat Q&A as question and answer resources.
- Use correct status codes:
    - 201 Created for new repo/question/job
    - 202 Accepted if you queue ingestion
    - 404 if repo/question not found
    - 422 for validation issues
    - 409 for conflicting ingestion states

The brief explicitly expects correct status and error codes.

Will the “AI bit” cause issues for he coursework is literally designed to encourage creative GenAI use and even mentions MCP as an advanced feature.

Just make sure you d* central**, not optional.
2) **Your API still works without the model being perfect** (you can always return citations, confidence, and clear error states).

Simple “extension path” for later (without risking the coursework)

Once you have the above working, later you can add:

- vulnerability findings resource: `GET /repos/{repoId}/findings`
- “explain file/function” endpoint: `POST /repos/{repoId}/explanations`
- MCP server wrapper so an agent can call your endpoints (this is literally named as an advanced feature band).

If you want, I can turn this into a vspec” for you next: database schema, exact endpoint list (minimum + stretch), and a demo plan for the 5 minute presentation that hits the marking grid cleanly.

ok based off this idea you suggested: 2. AI-Powered Code Intelligence & Search API,

what if i want to build an API where instead of searching the code, it analyses the repo and users should be able to ask questions about the repo. Does this qualify as a RESTful API as asked for our coursework? I know this is a good idea and theres a possibility of me extending this project to make it better later but yeah.

Look at this instagram reel below for more info on the idea I mentioned above.
https://www.instagram.com/reel/DUb-ECLj9XY/?igsh=MWZ2ZGFkNmwxd2poZA==

Feb 22

Searched the web
Searched the web
Perfect! I couldn't access the Instagram reel, but based on your description and the search results, I understand what you want: a Repository Question-Answering API (like RepoGPT or RepoChat). This is an EXCELLENT idea and absolutely qualifies as a RESTful API for your coursework!

✅ YES, This Qualifies as RESTful API - Here's Why:
RESTful Architecture Works Here:
Even though the experience feels conversational (chat-like), the underlying API structure is 100% RESTful. Here's how:

Your API Endpoints (RESTful CRUD):

```
POST   /api/repositories          # Create - Add a new repo to analyze
GET    /api/repositories          # Read - List all analyzed repos
GET    /api/repositories/{id}     # Read - Get specific repo details
```

```
PUT    /api/repositories/{id}        # Update - Re-analyze or update repo
DELETE /api/repositories/{id}        # Delete - Remove repo from system

POST   /api/repositories/{id}/questions  # Ask a question about the repo
GET    /api/repositories/{id}/questions  # Get question history
GET    /api/questions/{question_id}      # Get specific question/answer

POST   /api/repositories/{id}/analyze    # Trigger analysis job
GET    /api/repositories/{id}/analysis   # Get analysis status/results
```

This is RESTful because:

✅ Uses HTTP methods (GET, POST, PUT, DELETE)
✅ Resources identified by URIs
✅ Stateless (each request is independent)
✅ Returns JSON responses
✅ Standard HTTP status codes (200, 404, 500)

The Q&A feature is just ONE endpoint (POST /api/repositories/{id}/questions) that uses AI behind the scenes!

🎯 Your Project: "CodeQuery API" or "RepoMind API"

Core Concept:

A data-driven REST API that analyzes GitHub repositories and enables natural language questioning about codebases.

📋 Complete Technical Architecture

Phase 1: Repository Ingestion (CRUD - Create)

Endpoint: POST /api/repositories

Request:

```
json
{
  "repo_url": "https://github.com/username/repo",
  "branch": "main",
  "name": "My Project"
}
```

What Happens:

Clone the repository
Extract code files (.py, .js, .java, etc.)
Parse code structure (functions, classes, imports)
Generate embeddings using OpenAI/HuggingFace
Store vectors in database (PostgreSQL + pgvector)
Store metadata in relational tables

Database Tables:

```
sql
-- repositories table
```

```
CREATE TABLE repositories (
    id SERIAL PRIMARY KEY,
    name VARCHAR(255),
    repo_url TEXT,
    branch VARCHAR(100),
    status VARCHAR(50), -- 'analyzing', 'ready', 'failed'
    total_files INTEGER,
    total_lines INTEGER,
    languages JSON,  -- {"python": 45, "javascript": 30, ...}
    analyzed_at TIMESTAMP,
    created_at TIMESTAMP DEFAULT NOW()
);

-- code_chunks table (for embeddings)
CREATE TABLE code_chunks (
    id SERIAL PRIMARY KEY,
    repository_id INTEGER REFERENCES repositories(id),
    file_path TEXT,
    chunk_text TEXT,
    chunk_type VARCHAR(50),  -- 'function', 'class', 'docstring'
    line_start INTEGER,
    line_end INTEGER,
    embedding vector(1536),  -- pgvector extension
    created_at TIMESTAMP DEFAULT NOW()
);

-- questions table
CREATE TABLE questions (
    id SERIAL PRIMARY KEY,
    repository_id INTEGER REFERENCES repositories(id),
    question_text TEXT,
    answer_text TEXT,
    confidence_score FLOAT,
    sources JSON,  -- [{"file": "app.py", "line": 45}, ...]
    processing_time_ms INTEGER,
    created_at TIMESTAMP DEFAULT NOW()
);

-- analysis_metrics table
CREATE TABLE analysis_metrics (
    id SERIAL PRIMARY KEY,
    repository_id INTEGER REFERENCES repositories(id),
    metric_type VARCHAR(100),  -- 'complexity', 'test_coverage', etc.
    metric_value JSON,
    created_at TIMESTAMP DEFAULT NOW()
);
```

Phase 2: Question Answering (CRUD - Create for questions)

Endpoint: POST /api/repositories/{id}/questions

Request:

```
json
{
  "question": "How is authentication implemented in this codebase?"
}
```

What Happens (RAG Pipeline):

Question Embedding: Convert user question to vector
Similarity Search: Query pgvector for relevant code chunks

```
sql
   SELECT file_path, chunk_text, line_start
   FROM code_chunks
   WHERE repository_id = $1
   ORDER BY embedding <=> $question_embedding
   LIMIT 5;
```

Context Building: Gather top-k similar code chunks
LLM Query: Send to OpenAI/Anthropic with context:

```
python
   prompt = f"""
   Based on this codebase context:
   {retrieved_chunks}

   Answer this question: {user_question}
   Cite specific files and line numbers.
   """
```

Response Formatting: Return answer with citations
Response:

```
json
{
  "question_id": 123,
  "question": "How is authentication implemented?",
  "answer": "Authentication is implemented using JWT tokens. The main logic is in auth.py
where the authenticate_user() function validates credentials and generates tokens.",
  "confidence": 0.92,
  "sources": [
    {
      "file": "backend/auth.py",
      "line_start": 45,
      "line_end": 78,
      "relevance": 0.95
    },
    {
      "file": "middleware/jwt.py",
      "line_start": 12,
      "line_end": 34,
```

```
    "relevance": 0.87
  }
 ],
 "processing_time_ms": 1234
}
```

Phase 3: Other CRUD Operations

Read Operations:

```
http
GET /api/repositories                  # List all repos
GET /api/repositories/{id}             # Get repo details
GET /api/repositories/{id}/questions   # Question history
GET /api/repositories/{id}/analysis    # Code analysis results
GET /api/repositories/{id}/files       # File tree
GET /api/repositories/{id}/statistics  # Code stats
```

Update Operations:

```
http
PUT /api/repositories/{id}             # Re-analyze repo
POST /api/repositories/{id}/sync       # Pull latest changes
```

Delete Operations:

```
http
DELETE /api/repositories/{id}          # Remove repo and all data
DELETE /api/questions/{id}             # Delete question
```

🔧 Tech Stack Recommendation

```
yaml
Backend Framework:
  - FastAPI (perfect for your background, async support, auto-docs)

Database:
  - PostgreSQL 16+ (with pgvector extension for embeddings)

AI/ML:
  - OpenAI API (GPT-4o for Q&A)
  - HuggingFace Transformers (sentence-transformers for embeddings)
  - Alternative: Anthropic Claude API

Code Analysis:
  - tree-sitter (parse code syntax trees)
  - ast (Python's built-in for Python code)
  - git-python (repository operations)

Vector Search:
  - pgvector (PostgreSQL extension)
  - Alternative: FAISS, ChromaDB, Pinecone

Background Jobs:
```

- Celery + Redis (for async repo analysis)
- Alternative: FastAPI BackgroundTasks

Deployment:
- Backend: Railway.app (free tier, PostgreSQL included)
- Database: NeonDB (serverless Postgres with pgvector)
- Alternative: PythonAnywhere

🚀 How This Scores 80-90+ Grade
Content (75%) - Targeting 80-85%:
✅ Advanced CRUD Operations:

Not just simple CRUD, but intelligent data processing
Background job processing (repo analysis)
Complex queries (vector similarity search)
✅ Novel Technology:

pgvector integration (cutting-edge)
RAG (Retrieval-Augmented Generation) architecture
Code parsing and embeddings
✅ Professional Architecture:

Async processing with Celery
Proper separation of concerns
Error handling and retry logic
✅ API Documentation:

FastAPI auto-generates Swagger docs
Add custom examples and descriptions
✅ Testing:

Unit tests for each endpoint
Integration tests for Q&A pipeline
Mock LLM responses for testing
Presentation (15%) - Targeting 13-14%:
Demo Script:

Show API docs (Swagger UI)
POST a GitHub repo URL → Show analysis status
GET analysis results → Show code structure
POST a question → Live answer with citations
Show database with embeddings visualized
Q&A (10%) - Targeting 9-10%:
Expected Questions:

"Why did you choose pgvector over FAISS?"
"How do you handle large repositories?"
"What's your chunking strategy for code?"
"How do you ensure answer quality?"

💡 Advanced Features (Push to 90+)
1. MCP Server Implementation (Outstanding - 90+)
python
```
# Make your API MCP-compatible
# Let Claude Code/other agents use your API!

@app.post("/mcp/ask")
async def mcp_question_handler(repo_id: int, question: str):
    """MCP-compatible endpoint for AI agents"""
    # Return structured response for agent consumption
    pass
```
2. Code Visualization Endpoint
http
```
GET /api/repositories/{id}/graph
# Returns code dependency graph as JSON
```
3. Multi-Repository Search
http
```
POST /api/search/global
{
  "query": "How do authentication systems work?",
  "repositories": [1, 2, 3]  # Search across multiple repos
}
```
4. Semantic Code Search
http
```
POST /api/repositories/{id}/code-search
{
  "query": "function that validates email addresses",
  "file_types": [".py"]
}
# Returns actual code snippets, not just Q&A
```

📅 **8-Week Development Timeline**

Week 1-2: Foundation
- Set up FastAPI project structure
- PostgreSQL + pgvector setup
- Basic repository CRUD endpoints
- Git clone functionality

Week 3-4: Core Intelligence
- Code parsing (tree-sitter integration)
- Embedding generation pipeline
- Vector storage and indexing
- Basic Q&A endpoint (hardcoded responses)

Week 5-6: AI Integration
- OpenAI API integration
- RAG pipeline implementation
- Citation extraction
- Question history storage

Week 7: Polish & Testing
- Error handling
- API documentation
- Unit tests (pytest)
- Performance optimization

Week 8: Deployment & Presentation
- Deploy to Railway/PythonAnywhere
- Create demo repository
- Presentation slides
- Technical report writing

📝 **GenAI Usage Strategy (for 90+ band)**

Document these AI uses in your technical report:

1. Creative Problem-Solving:
```
"I used Claude to explore different vector database options:
- Asked: 'Compare pgvector, FAISS, and Pinecone for code embeddings'
- Explored trade-offs in latency, cost, scalability
- Decided on pgvector for simplicity and PostgreSQL integration"
```

2. Architecture Design:
```
"Used GPT-4 to validate my RAG pipeline design:
- Discussed chunking strategies for code
- Explored hybrid search (keyword + semantic)
- Implemented based on AI-suggested best practices"
```

3. Code Generation:
```
"Generated boilerplate for tree-sitter integration:
- Claude helped write parser for multiple languages
- Iterated on code structure extraction logic
- All code reviewed and understood before integration"
```
🎓 Why This Project is Perfect for You:
✅ Aligns with your experience: FastAPI, AI, databases

✅ Shows ML knowledge: Embeddings, RAG, vector search
✅ Practical for your degree: Use it for your final year project!
✅ Novel approach: Few students will attempt this
✅ Great demo: Impressive live Q&A on real codebases
✅ CV boost: "Built RAG system for code intelligence"
✅ Extensible: Can add features later (MCP, multi-repo, etc.)
🤔 Final Confirmation: This is 100% RESTful
The coursework says: "data-driven web API that connects to a database"

Your project:

✅ Data-driven (stores repos, questions, embeddings)
✅ Web API (RESTful HTTP endpoints)
✅ Database integration (PostgreSQL with complex queries)
✅ Full CRUD operations (on repos, questions, analysis)
The AI aspect is just the intelligence layer - the API structure is pure REST!

Want me to:

Generate the complete FastAPI project structure?
Write the database schema with pgvector setup?
Create a detailed step-by-step implementation guide?
Draft your technical report outline?
Design your presentation slides structure?
This project is chef's kiss 👨‍🍳💋 for your background and the coursework requirements!
Should we start building? 🚀

I want to use docker for this project

Use Docker from the start, but in a way that does not slow you down.

You will use it for two things:

1. run Postgres locally in one command
2. package your FastAPI app so the examiner can run it reliably

You can still develop normally on your machine. Docker just standardises the environment.

When to use Docker in this project

Phase 1 (Day 1): use Docker only for the database

This is the cleanest early win.

- You keep running FastAPI with `uvicorn` locally
- Postgres runs in a container
- You connect via `DATABASE_URL`

Why this is good:

- avoids installing Postgres on your laptop
- your dev setup is closer to deployment
- looks professional in your README and viva

Phase 2 (After CRUD works): containerise the API as well

Once `/repos` CRUD is working and migrations run, you add:

- a `Dockerfile` for the API
- `docker-compose.yml` to run API + Postgres together

Why not earlier:

- containerising too early can slow debugging
- but once your API is stable, Docker makes everything reproducible

Phase 3 (Final week): demo and submission polish

You will demo with:

- `docker compose up`
- open Swagger UI
- run your demo flow

This makes the viva smooth because everything starts the same way every time.

What you need to add to your repo

1) docker-compose.yml (API + Postgres)

Put this in the project root.

```
services:
 db:
   image: postgres:16
   container_name: repolens_db
   environment:
     POSTGRES_USER: repolens
```

```
    POSTGRES_PASSWORD: repolens
    POSTGRES_DB: repolens
  ports:
    - "5432:5432"
  volumes:
    - repolens_pgdata:/var/lib/postgresql/data
  healthcheck:
    test: ["CMD-SHELL", "pg_isready -U repolens -d repolens"]
    interval: 5s
    timeout: 5s
    retries: 10

 api:
  build: .
  container_name: repolens_api
  environment:
    DATABASE_URL: postgresql+psycopg://repolens:repolens@db:5432/repolens
    # Put your model keys in a .env file, not in this YAML
    # OPENAI_API_KEY: ${OPENAI_API_KEY}
  ports:
    - "8000:8000"
  depends_on:
    db:
      condition: service_healthy

volumes:
 repolens_pgdata:
```

2) Dockerfile (containerise FastAPI)

Also in the project root.

```
FROM python:3.11-slim

WORKDIR /app

# System deps (git is needed to clone repos during ingestion)
RUN apt-get update && apt-get install -y --no-install-recommends \
  git \
&& rm -rf /var/lib/apt/lists/*

# Install python deps
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

# Copy app
COPY . .
```

```
# Expose and run
EXPOSE 8000
CMD ["uvicorn", "app.main:app", "--host=0.0.0.0", "--port=8000"]
```

3) .env (keep secrets out of git)

Create a `.env` file in the root:

```
OPENAI_API_KEY=your_key_here
```

Then in `docker-compose.yml`, add:

```
  env_file:
    - .env
```

Also add `.env` to `.gitignore`.

How to run it (your new standard workflow)

Start everything

```
docker compose up --build
```

Open Swagger

- http://localhost:8000/docs

Stop

```
docker compose down
```

Reset the database (careful, deletes data)

```
docker compose down -v
```

How migrations fit with Docker (Alembic)

You have two clean options. Pick whichever you find easier.

Option A (simplest): run migrations from inside the API container

Once containers are up:

```
docker compose exec api alembic upgrade head
```

This is ideal for your README because the examiner can copy paste it.

Option B: run migrations automatically on container start (later)

You can add an entrypoint script, but do this only after everything works.

Where to store cloned repos inside Docker

Your ingestion clones repos to a workspace folder like `/app/workspace`.

In Docker, that folder disappears when the container is removed unless you mount a volume.

For coursework, it is fine to keep it ephemeral, but if you want persistence, add:

```
volumes:
  - ./workspace:/app/workspace
```

This makes debugging easier too because you can inspect the cloned repo on your machine.

how do i test this manually?

Here's a clean manual test plan you can follow **right now**, without guessing, and without downloading anything big.

0) Start lightweight mode

From project root:

```
docker compose up --build -d
docker compose ps
```

You want to see **api + db** only (and no ollama).

---

1) Open Swagger and find the exact routes

Open:

- http://localhost:8000/docs

In the left sidebar, note the actual paths. Your app might use:

- `/repositories` or `/api/v1/repositories`
- `/analysis/health` or `/health`

Use the paths you see there.

---

2) Manual CRUD test (Repositories)

A) Create a repo (use a tiny one)

In Swagger, go to `POST /repositories` (or equivalent) and use:

```
{
 "provider": "github",
 "url": "https://github.com/octocat/Hello-World",
 "default_branch": "master"
}
```

Expected:

- **201 Created**
- JSON response includes an `id`

Write down the returned `id`.

B) List repos

Call `GET /repositories`

Expected:

- **200 OK**
- Your repo appears in the list

C) Get repo by id

Call `GET /repositories/{id}` using the id you saved

Expected:

- **200 OK**
- JSON matches the repo you created

D) Update repo

Find your update endpoint:

- `PUT /repositories/{id}` or `PATCH /repositories/{id}`

If PATCH exists, use:

```
{
 "default_branch": "main"
}
```

If there is an “action” field and you want to trigger reanalyse, you may see something like:

```
{
 "action": "reanalyze"
}
```

Expected:

- **200 OK**
- `default_branch` updated OR status becomes pending if reanalyze

E) Delete repo

Call `DELETE /repositories/{id}`

Expected:

- **204 No Content**

Then confirm it is gone:

- `GET /repositories/{id}` should return **404**

---

3) Negative tests (status codes matter for marks)

A) 404 test

Call `GET /repositories/{id}` with a random id that doesn’t exist.

Expected:

- **404 Not Found**
- JSON error response (whatever your API standard is)

B) 422 validation test

Try creating a repo with missing url:

```
{
 "provider": "github"
}
```

Expected:

- **422 Unprocessable Entity** (FastAPI validation error)

---

4) Manual ingestion test (storage-safe)

Important

Only do this if your system ingests by cloning. Use **octocat/Hello-World** only.

How ingestion is triggered depends on your routes. You likely have one of these:

- Auto-ingest triggered on `POST /repositories` (you mentioned it queues ingestion)
- A dedicated endpoint like `POST /repositories/{id}/ingest`
- Update endpoint with `action="reanalyze"`

If ingest is auto-triggered on create

After creating the repo, wait ~10–30 seconds then check:

- `GET /repositories/{id}`

Look for fields like:

- `status` changing from `pending` → `ready`/`completed`
- an `analysis_job_id` or similar

If ingest is a dedicated endpoint

Call it for that repo id and look for:

- **202 Accepted** or **200 OK**
- a job id or “local-task” returned

---

5) Manual Q&A test (with lightweight fallback)

Go to `POST /questions` or `POST /repositories/{id}/questions` and ask something simple:

```
{
 "question": "What files exist in this repo and what do they do?"
}
```

Expected (in lightweight mode, no ollama):

- It should **not crash**
- You should get either:
  - a normal response with `sources` filled
  - OR a 503 with `sources_retrieved` (based on your current implementation)

If you still see a 503, check that the response contains sources. That proves retrieval works even without LLM.

---

6) Quick sanity checks from Docker logs (very useful)

If something seems stuck:

```
docker compose logs --tail=80 api
```

Look for:

- “Queued analysis… local-task”
- clone path
- chunk count
- any exception stack trace

---

7) If you prefer curl instead of Swagger

If your endpoints are exactly `/repositories`, here are templates:

Create

```
curl -s -X POST "http://localhost:8000/repositories" \
 -H "Content-Type: application/json" \
 -d
'{"provider":"github","url":"https://github.com/octocat/Hello-World","default_branch":"master"}'
```

List

```
curl -s "http://localhost:8000/repositories"
```

Get by id

```
curl -s "http://localhost:8000/repositories/<ID>"
```

Update

```
curl -s -X PATCH "http://localhost:8000/repositories/<ID>" \
 -H "Content-Type: application/json" \
 -d '{"default_branch":"main"}'
```

Delete

```
curl -i -X DELETE "http://localhost:8000/repositories/<ID>"
```

What to report back to me (so I can guide the next fix fast)

After you do the manual run, send me:

1. The exact base paths shown in Swagger (eg `/repositories` vs `/api/v1/repositories`)
2. The response from creating a repo (just the key fields like id, status)
3. Whether status changes after ingestion
4. What the Q&A endpoint returns in lightweight mode (200 vs 503, and whether sources appear)

With that, I'll tell you the next best code change (likely: make Q&A return 200 with a warning instead of raising 503, and add a retrieval-only endpoint for a rock-solid demo).